

3.4 仿真准备与机器人描述包

所属阶段： 第三阶段 · 坐标变换与机器人建模

讲次数量： 12 节

预计时长： ~90 分钟

前置依赖： 3.3 Xacro 进阶建模

学习目标

完成本节后，学员将能够：

- 在 URDF/Xacro 中为参与物理仿真的实体 Link 添加 Collision 和 Inertial 标签
- 使用 Xacro 宏计算实心几何体的惯性张量，并修复不合理的惯性值
- 配置 Gazebo 材质颜色，使仿真视觉与 RViz 一致
- 创建标准的 `my_robot_description` 功能包，组织 `urdf / launch / rviz` 目录
- 编写 Launch 文件，一键启动 `robot_state_publisher` + RViz 2 显示机器人模型
- 理解 `robot_description` 参数的传递链路，为第四阶段 Gazebo 仿真做好准备

本节配套源码

本节继续使用 `stage3_code/my_robot_description`，在 3.3 的模块化 Xacro 基础上新增和修改：

```
stage3_code/my_robot_description/  
├─ urdf/my_robot.urdf.xacro  
├─ urdf/my_robot/  
│   ├── inertia_macros.xacro  
│   ├── gazebo_materials.xacro  
│   ├── base.xacro  
│   ├── sensors.xacro  
│   └─ macros.xacro  
├─ urdf/exercise_properties.urdf.xacro  
├─ launch/display.launch.py  
├─ launch/minimal_display.launch.py  
└─ rviz/my_robot.rviz
```

本节源码已按 Jazzy 环境验证：主模型展开后包含 6 个 Link、5 个 Joint、5 个 `<collision>`、5 个 `<inertial>` 和 5 个 `<gazebo reference=...>`；其中 `base_footprint` 是无体积的虚拟参考系，不配置物理几何。练习模型展开后包含 3 个 `<collision>` 和 3 个 `<inertial>`。

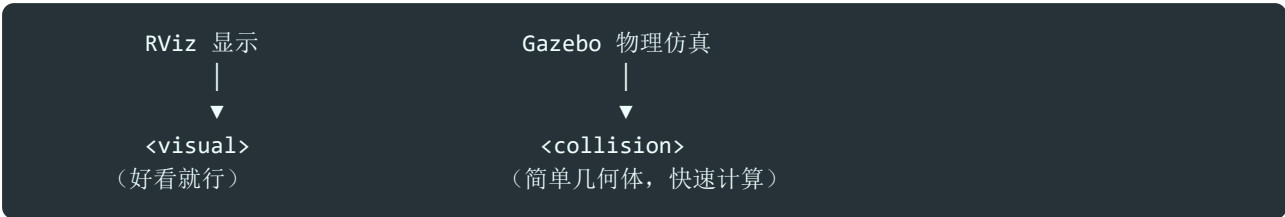
讲次 1：在 URDF 中添加碰撞标签 (Collision Tags)

知识点

3.2 ~ 3.3 中我们只定义了 `<visual>` 标签——它告诉 RViz "机器人看起来什么样"。但 Gazebo Harmonic 物理引擎还需要知道"机器人碰撞边界在哪里"，这由 `<collision>` 标签描述。

Visual vs Collision

标签	用途	谁使用	精度要求
<code><visual></code>	外观渲染	RViz、Gazebo 渲染器	可用精细 mesh
<code><collision></code>	物理碰撞边界	Gazebo 物理引擎	越简单越好 (性能)



Collision 基本结构

```
<link name="base_link">
  <visual>
    <!-- 外观: 可以是复杂 mesh -->
  </visual>
  <collision>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <geometry>
      <box size="0.178 0.138 0.109"/>
    </geometry>
  </collision>
</link>
```

为 my_robot 底盘添加 Collision

在 `base.xacro` 或主 xacro 文件的 `base_link` 中:

```

<link name="base_link">
  <visual>
    <origin xyz="0 0 ${base_height / 2.0}" rpy="0 0 0"/>
    <geometry>
      <box size="${base_length} ${base_width} ${base_height}"/>
    </geometry>
    <material name="blue"/>
  </visual>
  <collision>
    <origin xyz="0 0 ${base_height / 2.0}" rpy="0 0 0"/>
    <geometry>
      <box size="${base_length} ${base_width} ${base_height}"/>
    </geometry>
  </collision>
</link>

```

为轮子添加 Collision

修改 `macros.xacro` 中的 `drive_wheel` 宏:

```

<xacro:macro name="drive_wheel" params="prefix y_reflect">
  <link name="${prefix}_wheel_link">
    <visual>
      <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
      <geometry>
        <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
      </geometry>
      <material name="black"/>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
      <geometry>
        <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
      </geometry>
    </collision>
  </link>
  <!-- joint 定义不变 -->
</xacro:macro>

```

为激光雷达添加 Collision

```
<link name="base_scan">
  <visual>
    <geometry>
      <cylinder radius="${scan_radius}" length="${scan_height}"/>
    </geometry>
    <material name="black"/>
  </visual>
  <collision>
    <geometry>
      <cylinder radius="${scan_radius}" length="${scan_height}"/>
    </geometry>
  </collision>
</link>
```

Collision 设计原则

1. **简单优先**: 用 box / cylinder / sphere, 避免在 collision 中使用 mesh
2. **略小于 visual**: 可防止 mesh 细节导致碰撞抖动 (可选优化)
3. **成对出现**: 每个需要物理交互的 link 都应有 collision
4. **fixed 关节的从 link**: 如 `base_footprint` 通常不需要 collision (它是虚拟参考系)

不需要 Collision 的 Link

Link	原因
<code>base_footprint</code>	虚拟地面投影点, 无体积
纯视觉装饰件	不影响运动学和碰撞 (可选省略)

验证

```
xacro my_robot.urdf.xacro | grep -c '<collision>'
# 预期: 5 (base_link, base_scan, left_wheel, right_wheel, back_caster)
```

关键点

- `<collision>` 定义物理碰撞边界, Gazebo Harmonic 仿真的必要标签
- Collision 几何体应简单 (box/cylinder/sphere), 与 visual 可不同
- 每个参与物理交互的 link 都需要 collision
- `base_footprint` 等虚拟 link 不需要 collision
- 轮子的 collision 同样需要 `ropy="${pi/2} 0 0"` 旋转以匹配圆柱朝向

讲次 2: 【Activity】添加 Collision Tags + 解答

知识点

练习目标

为 3.3 练习中的 `exercise_properties.urdf.xacro` 添加完整的 Collision 标签。

练习要求

1. 为 `base_link`、`left_wheel_link`、`right_wheel_link` 各添加一个 `<collision>` 块
2. Collision 几何体与 Visual 一致（同尺寸、同旋转）
3. 所有尺寸仍通过 Properties 引用，不允许硬编码
4. `base_footprint` 不添加 collision
5. 验证：`xacro` 展开后 `grep -c '<collision>'` 输出 3

验证命令

```
xacro exercise_properties.urdf.xacro | grep -c '<collision>'
# 预期: 3

xacro exercise_properties.urdf.xacro | grep -A5 'left_wheel_link' | grep collision
# 预期: 包含 <collision> 块
```

参考答案 (关键片段)

在 `base_link` 的 `</visual>` 之后添加:

```
<collision>
  <origin xyz="0 0 ${chassis_height / 2.0}" rpy="0 0 0"/>
  <geometry>
    <box size="${chassis_length} ${chassis_width} ${chassis_height}"/>
  </geometry>
</collision>
```

在 `left_wheel_link` 的 `</visual>` 之后添加:

```
<collision>
  <origin xyz="0 0 0" rpy="${pi/2} 0 0"/>
  <geometry>
    <cylinder radius="${wheel_radius}" length="${wheel_width}"/>
  </geometry>
</collision>
```

`right_wheel_link` 同理。

常见错误

错误	现象	修复
轮子 collision 缺少 rpy 旋转	Gazebo 中轮子碰撞方向错误	添加 <code>rpy="\${pi/2} 0 0"</code>
collision 尺寸硬编码	改 property 后碰撞不更新	全部用 <code>\${}</code> 引用
base_footprint 加了 collision	地面出现莫名碰撞体	删除 base_footprint 的 collision

关键点

- 练习验证 collision 与 visual 配对、参数化一致
 - 轮子 collision 的 origin 旋转是关键细节
 - 用 `grep -c '<collision>'` 快速检查覆盖率
-

讲次 3：在 URDF 中添加惯性标签 (Inertial Tags)

知识点

Gazebo Harmonic 的物理引擎需要参与动力学求解的实体 link 具有**质量 (mass)** 和**惯性张量 (inertia)**，才能正确模拟重力、加速和碰撞响应。这些信息由 `<inertial>` 标签提供；`base_footprint` 这类纯参考 frame 不属于实体 link。

Inertial 基本结构

```
<inertial>
  <origin xyz="0 0 0" rpy="0 0 0"/>  <!-- 质心位置 -->
  <mass value="0.826"/>                <!-- 质量 (kg) -->
  <inertia ixx="..." ixy="0" ixz="0"
            iyy="..." iyz="0" izz="..." /> <!-- 惯性张量 (kg·m²) -->
</inertial>
```

惯性张量物理含义

对于刚体，惯性张量描述质量分布——决定了机器人受力时的旋转响应：

$$I = \begin{bmatrix} i_{xx} & i_{xy} & i_{xz} \\ i_{xy} & i_{yy} & i_{yz} \\ i_{xz} & i_{yz} & i_{zz} \end{bmatrix}$$

- **对角元素** (i_{xx}, i_{yy}, i_{zz})：绕 x/y/z 轴的转动惯量
- **非对角元素** (i_{xy}, i_{xz}, i_{yz})：质量不对称时为非零，对称几何体通常为 0

实心长方体惯性公式

对于质量为 m 、尺寸为 $a \times b \times c$ 的长方体（质心在原点）：

$$\begin{aligned} i_{xx} &= m/12 \times (b^2 + c^2) \\ i_{yy} &= m/12 \times (a^2 + c^2) \\ i_{zz} &= m/12 \times (a^2 + b^2) \end{aligned}$$

实心圆柱体惯性公式

对于质量为 m 、半径为 r 、高度为 h 的圆柱（轴沿 z 轴）：

$$\begin{aligned} i_{xx} &= i_{yy} = m/12 \times (3r^2 + h^2) \\ i_{zz} &= m/2 \times r^2 \end{aligned}$$

当圆柱轴沿 x 轴时（轮子的情况），需根据旋转调整。

用 Xacro 计算惯性

在 `properties.xacro` 中添加：

```
<!-- 底盘惯性（长方体公式） -->
<xacro:property name="base_ixx" value="${base_mass / 12.0 * (base_width*base_width + base_height*base_height)}"/>
<xacro:property name="base_iyy" value="${base_mass / 12.0 * (base_length*base_length + base_height*base_height)}"/>
<xacro:property name="base_izz" value="${base_mass / 12.0 * (base_length*base_length + base_width*base_width)}"/>

<!-- 轮子惯性（圆柱公式，轴沿 y 轴） -->
<xacro:property name="wheel_ixx" value="${wheel_mass / 12.0 * (3*wheel_radius*wheel_radius + wheel_length*wheel_length)}"/>
<xacro:property name="wheel_iyy" value="${wheel_mass / 2.0 * wheel_radius*wheel_radius}"/>
<xacro:property name="wheel_izz" value="${wheel_mass / 12.0 * (3*wheel_radius*wheel_radius + wheel_length*wheel_length)}"/>
```

为 base_link 添加 Inertial

```
<link name="base_link">
  <inertial>
    <origin xyz="0 0 ${base_height / 2.0}" rpy="0 0 0"/>
    <mass value="${base_mass}"/>
    <inertia ixx="${base_ixx}" ixy="0" ixz="0"
            iyy="${base_iyy}" iyz="0" izz="${base_izz}"/>
  </inertial>
  <visual>...</visual>
  <collision>...</collision>
</link>
```

TurtleBot3 Burger 参考值

Link	质量 (kg)	ixx	iyy	izz
base_link	0.826	2.21e-3	2.12e-3	2.01e-3
wheel_link	0.0285	1.12e-5	1.12e-5	2.07e-5
base_scan	0.114	1.0e-3	1.0e-3	1.0e-3

注意轮子的坐标系：官方 URDF 中轮子 `<inertial>` 的 `origin` 带 `rpy="1.57 0 0"` 旋转，惯性张量在旋转后的坐标系中表达，因此**轴向分量是 izz** (2.07e-5)。本课模型的轮子 `inertial origin` 不旋转、轮轴沿 y 方向，轴向分量是 **iyy**——对比数值时注意换算。

公式计算值与官方 URDF 接近但不完全相同（官方可能包含更精确的质量分布），后续讲次将讲解如何修复。

Link 标签推荐顺序

```
<link name="...">
  <inertial>...</inertial>    <!-- 1. 物理属性 -->
  <visual>...</visual>       <!-- 2. 外观 -->
  <collision>...</collision> <!-- 3. 碰撞 -->
</link>
```

关键点

- `<inertial>` 提供质量与惯性张量，Gazebo 物理仿真的硬性要求
 - 实心几何体可用公式计算惯性，Xacro 支持在 property 中写公式
 - 对称物体的非对角元素 (ixy, ixz, iyz) 设为 0
 - 质量参数应参考真实机器人 (TurtleBot3 Burger 是好的基准)
 - Link 内标签顺序推荐: inertial → visual → collision
-

讲次 4: 【Activity】Inertia Macros 练习与解答

知识点

练习目标

创建一个惯性计算宏 `inertia_box`，封装长方体惯性公式，并在练习机器人中应用。

练习要求

1. 在 `exercise_properties.urdf.xacro` (或单独的 `inertia_macros.xacro`) 中定义宏:

```
<xacro:macro name="inertia_box" params="mass x y z">
  <!-- 根据长方体公式计算并输出 <inertia .../> 标签 -->
</xacro:macro>
```

1. 在 `base_link` 的 `<inertial>` 块中调用该宏
2. 为两个轮子手动使用圆柱公式添加 `<inertial>` (为下一讲的宏化做准备)
3. 验证: 展开后 `base_link` 的 `ixx` 值约为 $0.826/12 \times (0.138^2 + 0.109^2) \approx 2.13e-3$

参考答案

惯性宏定义:

```
<xacro:macro name="inertia_box" params="mass x y z">
  <inertia
    ixx="{mass / 12.0 * (y*y + z*z)}"
    ixy="0" ixz="0"
    iyy="{mass / 12.0 * (x*x + z*z)}"
    iyz="0"
    izz="{mass / 12.0 * (x*x + y*y)}"/>
</xacro:macro>

<!-- 圆柱轴沿 link 的 y 轴（侧装轮子），故 iyy 为轴向分量 -->
<xacro:macro name="inertia_cylinder_y" params="mass radius length">
  <inertia
    ixx="{mass / 12.0 * (3*radius*radius + length*length)}"
    ixy="0" ixz="0"
    iyy="{mass / 2.0 * radius*radius}"
    iyz="0"
    izz="{mass / 12.0 * (3*radius*radius + length*length)}"/>
</xacro:macro>
```

在 `base_link` 中使用:

```

<xacro:property name="chassis_mass" value="0.826"/>

<link name="base_link">
  <inertial>
    <origin xyz="0 0 ${chassis_height / 2.0}" rpy="0 0 0"/>
    <mass value="${chassis_mass}"/>
    <xacro:inertia_box mass="${chassis_mass}"
                      x="${chassis_length}"
                      y="${chassis_width}"
                      z="${chassis_height}"/>
  </inertial>
  <!-- visual, collision -->
</link>

```

在轮子 link 中使用：

```

<xacro:property name="wheel_mass" value="0.0285"/>

<link name="left_wheel_link">
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="${wheel_mass}"/>
    <xacro:inertia_cylinder_y mass="${wheel_mass}"
                             radius="${wheel_radius}"
                             length="${wheel_width}"/>
  </inertial>
  <!-- visual, collision -->
</link>

```

验证计算

```

python3 -c "
m, a, b, c = 0.826, 0.178, 0.138, 0.109
ixx = m/12 * (b**2 + c**2)
iyy = m/12 * (a**2 + c**2)
izz = m/12 * (a**2 + b**2)
print(f'ixx={ixx:.4e}, iyy={iyy:.4e}, izz={izz:.4e}')
"
# 预期: ixx=2.1287e-03, iyy=2.9987e-03, izz=3.4918e-03

```

关键点

- 惯性宏将物理公式封装为可复用模板，避免手动计算错误
- `inertia_box` 和 `inertia_cylinder_y` 是移动机器人最常用的两个惯性宏
- 宏输出的是 `<inertia>` 标签内容（不含 `<inertial>` 外层），嵌入到 `<inertial>` 块中使用
- 公式计算值应与 TurtleBot3 官方值在同一数量级

讲次 5：修复 URDF 惯性值

知识点

即使使用了公式计算，Gazebo 仿真中仍可能出现机器人**弹起、抖动、穿透地面**等现象。惯性参数是重点检查项之一，但还要同时排查碰撞体重叠、关节约束、控制增益、仿真时间步长和接触参数。本节聚焦惯性这一支诊断。

常见惯性问题

问题	现象	原因
质量与尺寸不匹配	碰撞和加速响应与实物差异大	只看绝对数值，未按材料和几何估算
惯性相对质量和尺寸过小	轻微碰撞就产生过大角加速度	惯性张量与几何尺寸不成比例
惯性张量非正定	模型加载失败或动力学异常	对角值非正、非对角值不合理
质心偏移错误	机器人倾斜或翻倒	<code><origin></code> 位置不对
总质量不合理	运动不符合预期	各 link 质量之和偏差太大

检查工具

```
# 检查 URDF 完整性
check_urdf my_robot.urdf

# 用 Gazebo 加载测试（第四阶段详细讲解，此处先验证 URDF）
xacro my_robot.urdf.xacro > /tmp/my_robot.urdf
```

修复策略 1：确保最小惯性值

对于极轻且极小的 link（如万向轮 caster），按公式得到的惯性可能使数值求解更敏感。本课模型使用经仿真验证的保守值作为教学配置：

```
<!-- 标准 xacro 不支持 max(), 对极轻量 link 直接设保守固定值 -->
<xacro:macro name="inertia_cylinder" params="mass radius length">
  <inertia
    ixx="1e-5"
    ixy="0" ixz="0"
    iyy="1e-5"
    iyz="0"
    izz="1e-5"/>
</xacro:macro>
```

注意： `1e-5` 不是通用下限，也不能脱离质量和尺寸直接套用。这里是为小型教学机器人选取的数值稳定性折中，会牺牲一部分物理真实性。真实项目应优先根据 CAD、实测质量或基本几何公式建模，再通过接

触和控制实验验证。标准 xacro 不支持直接调用 `max()`，需要时可以在外部脚本中预处理。

修复策略 2：为万向轮设置保守值

```
<xacro:property name="caster_mass" value="0.005"/>

<link name="back_caster_link">
  <inertial>
    <origin xyz="0 0 0" rpy="0 0 0"/>
    <mass value="${caster_mass}"/>
    <inertia ixx="1e-5" ixy="0" ixz="0"
            iyy="1e-5" iyz="0" izz="1e-5"/>
  </inertial>
  <visual>
    <geometry>
      <sphere radius="0.005"/>
    </geometry>
    <material name="black"/>
  </visual>
  <collision>
    <geometry>
      <sphere radius="0.005"/>
    </geometry>
  </collision>
</link>
```

修复策略 3：对齐 TurtleBot3 官方值

当公式计算值与仿真行为不理想时，可直接采用经过验证的官方值：

```
<!-- base_link 惯性：采用 TurtleBot3 Burger 官方值 -->
<link name="base_link">
  <inertial>
    <origin xyz="-0.032 0 0.070" rpy="0 0 0"/>
    <mass value="0.826"/>
    <inertia ixx="0.002212" ixy="0" ixz="0"
            iyy="0.002119" iyz="0" izz="0.002006"/>
  </inertial>
</link>
```

注意官方 URDF 中 `base_link` 的质心 `<origin xyz="-0.032 0 0.070">` 不在几何中心——这是因为 TurtleBot3 的电池和电路板分布不均匀。

修复策略 4: 总质量检查

```
# 快速估算总质量
python3 -c "
masses = {'base': 0.826, 'scan': 0.114, 'wheel_x2': 0.057, 'caster': 0.005}
total = sum(masses.values())
print(f'Total mass: {total:.3f} kg')
"
# 预期: ~1.002 kg (与 TurtleBot3 Burger ~1kg 一致)
```

修复检查清单

- [] 每个有 collision 的 link 都有 `<inertial>`
- [] 惯性张量对称、正定, 且量级与各 link 的质量和尺寸一致
- [] 总质量与目标实物一致 (本课小车约 1 kg)
- [] 质心位置合理 (不在 link 几何体外)
- [] `check_urdf` 无 error

关键点

- Gazebo 中弹起或抖动时, 应联合检查惯性、碰撞体、约束、控制器和仿真步长
 - 极轻量 link 可能需要经实验验证的保守惯性值, 但不存在适用于所有模型的固定下限
 - 可参考 TurtleBot3 官方 URDF 中经过仿真验证的惯性参数
 - 质心 `<origin>` 偏移反映真实质量分布, 不必强制在几何中心
 - 总质量应接近真实机器人 (Burger ~1 kg)
-

讲次 6：修复 Gazebo 材质颜色

知识点

RViz 通过 URDF 的 `<material><color rgba="..."></material>` 显示颜色，但 Gazebo 有独立的材质系统——如果不在 URDF 中额外配置 Gazebo 标签，仿真中机器人可能显示为灰色或白色。

Gazebo Classic vs Harmonic: 本节 `<gazebo>` 标签中的 `<material>`、`<mu1>` / `<mu2>` 摩擦系数写法面向 **Gazebo Classic** 兼容模式。Gazebo Harmonic (Ignition) 使用不同的 SDF 材质与物理配置方式，将在第四阶段专门讲解。现阶段在 RViz 中验证 visual 颜色即可。

Gazebo 材质配置方式

在 URDF 中嵌入 `<gazebo>` 标签（由 `gazebo_ros_pkgs` / `ros_gz_sim` 转换）：

```
<gazebo reference="base_link">
  <material>
    <ambient>0.0 0.0 0.8 1.0</ambient>
    <diffuse>0.0 0.0 0.8 1.0</diffuse>
    <specular>0.1 0.1 0.1 1.0</specular>
  </material>
</gazebo>
```

创建 Gazebo 材质文件

```
urdf/my_robot/gazebo_materials.xacro
```

```

<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">

  <!-- 底盘：蓝色 -->
  <gazebo reference="base_link">
    <material>
      <ambient>${color_blue}</ambient>
      <diffuse>${color_blue}</diffuse>
      <specular>0.1 0.1 0.1 1.0</specular>
    </material>
  </gazebo>

  <!-- 轮子：黑色 -->
  <gazebo reference="left_wheel_link">
    <material>
      <ambient>${color_black}</ambient>
      <diffuse>${color_black}</diffuse>
      <specular>0.05 0.05 0.05 1.0</specular>
    </material>
  </gazebo>

  <gazebo reference="right_wheel_link">
    <material>
      <ambient>${color_black}</ambient>
      <diffuse>${color_black}</diffuse>
      <specular>0.05 0.05 0.05 1.0</specular>
    </material>
  </gazebo>

  <!-- 激光雷达：深灰 -->
  <gazebo reference="base_scan">
    <material>
      <ambient>${color_dark_grey}</ambient>
      <diffuse>${color_dark_grey}</diffuse>
      <specular>0.3 0.3 0.3 1.0</specular>
    </material>
  </gazebo>

  <!-- 万向轮：黑色 -->
  <gazebo reference="back_caster_link">
    <material>
      <ambient>${color_black}</ambient>
      <diffuse>${color_black}</diffuse>
      <specular>0.05 0.05 0.05 1.0</specular>
    </material>
  </gazebo>

</robot>

```

仓库中的完整版本还为左右轮和万向轮追加了 μ_1 / μ_2 / k_p / k_d 摩擦与接触参数（见下文“摩擦系数配置”），此处先聚焦材质颜色。

在主文件中 include:

```
<xacro:include filename="my_robot/gazebo_materials.xacro"/>
```

摩擦系数配置（预置，第四阶段使用）

Gazebo Harmonic 中轮子的摩擦系数影响差速驱动的抓地力：

```
<gazebo reference="left_wheel_link">
  <mu1>1.0</mu1>
  <mu2>1.0</mu2>
  <kp>1000000.0</kp>
  <kd>100.0</kd>
  <material>
    <ambient>${color_black}</ambient>
    <diffuse>${color_black}</diffuse>
  </material>
</gazebo>
```

参数	含义	推荐值（橡胶轮）
mu1	主摩擦方向系数	1.0
mu2	次摩擦方向系数	1.0
kp	接触刚度	1e6
kd	接触阻尼	100

RViz vs Gazebo 颜色对照

环境	配置位置	格式
RViz 2	<code><material><color rgba="R G B A"/></material></code>	0.0 ~ 1.0
Gazebo Harmonic	<code><gazebo><material><ambient>...</ambient></material></gazebo></code>	0.0 ~ 1.0

两者使用相同的 RGBA 值即可保持一致。

验证方法

第四阶段启动 Gazebo 后目视检查。现阶段可在 RViz 中确认 visual 颜色正确：

```
ros2 launch my_robot_description display.launch.py
# 在 RViz 中检查：底盘蓝色、轮子黑色、雷达深灰
```

关键点

- Gazebo 颜色通过 `<gazebo reference="link_name">` 标签单独配置
- `ambient` 和 `diffuse` 设为主要颜色，`specular` 控制高光

- 轮子需配置摩擦系数 (μ_1/μ_2) , 第四阶段差速驱动依赖此参数
 - 颜色值与 RViz 保持一致, 使用相同的 RGBA Property
 - Gazebo 材质配置放在独立的 `gazebo_materials.xacro` 中
-

讲次 7：创建 Robot Description 功能包

知识点

将零散的 xacro 文件组织为标准的 ROS 2 **Robot Description 功能包**——这是 ROS 社区公认的机器人模型存放方式，也是后续 Gazebo、Nav2、MoveIt 2 集成的入口。

创建功能包

注意：若你已在 3.2 中创建了 `my_robot_description` 包，**跳过** `ros2 pkg create`，直接在现有包中补充目录和文件即可。

```
cd ~/ros2_ws/src

# 仅当包尚不存在时执行：
ros2 pkg create my_robot_description \
  --build-type ament_cmake \
  --dependencies xacro robot_state_publisher rviz2 joint_state_publisher_gui
```

标准目录结构

```
cd my_robot_description
mkdir -p urdf/my_robot launch rviz meshes
```

最终结构：

```
my_robot_description/
├── CMakeLists.txt
├── package.xml
├── urdf/
│   ├── my_robot.urdf.xacro          # 主入口
│   └── my_robot/
│       ├── properties.xacro
│       ├── materials.xacro
│       ├── macros.xacro
│       ├── gazebo_materials.xacro
│       └── inertia_macros.xacro
├── launch/
│   └── display.launch.py            # 后续讲次创建
├── rviz/
│   └── my_robot.rviz                # 后续讲次创建
└── meshes/                          # 可选：3D 模型文件
```

package.xml

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd" schematypens="http://www.w
<package format="3">
  <name>my_robot_description</name>
  <version>0.1.0</version>
  <description>URDF/Xacro description for my differential-drive mobile robot</description>
  <maintainer email="student@example.com">Student</maintainer>
  <license>Apache-2.0</license>

  <buildtool_depend>ament_cmake</buildtool_depend>

  <depend>ament_index_python</depend>
  <depend>launch</depend>
  <depend>launch_ros</depend>
  <depend>xacro</depend>
  <depend>robot_state_publisher</depend>
  <depend>joint_state_publisher_gui</depend>
  <depend>rviz2</depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_description)

find_package(ament_cmake REQUIRED)

# 安装 urdf、launch、rviz、meshes 目录到 share/
install(DIRECTORY urdf launch rviz meshes
  DESTINATION share/${PROJECT_NAME}
)

ament_package()
```

构建与验证

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash

# 验证文件已安装
ros2 pkg prefix my_robot_description
# 输出: ~/ros2_ws/install/my_robot_description

ls install/my_robot_description/share/my_robot_description/urdf/
# 预期: my_robot.urdf.xacro my_robot/
```

Robot Description 包的职责

职责	文件位置
机器人结构描述	<code>urdf/*.xacro</code>
启动可视化	<code>launch/display.launch.py</code>
RViz 配置	<code>rviz/my_robot.rviz</code>
3D 模型资源	<code>meshes/*.stl</code> / <code>*.dae</code>

命名惯例： 功能包名以 `_description` 结尾（如 `turtlebot3_description` 、`my_robot_description` ），存放模型的"描述"，不包含控制逻辑。

关键点

- `my_robot_description` 是标准 ROS 2 机器人描述包
 - 目录结构: `urdf` / `launch` / `rviz` / `meshes` 四大目录
 - `CMakeLists.txt` 用 `install(DIRECTORY ...)` 安装资源到 `share/`
 - `package.xml` 声明 `xacro`、`robot_state_publisher`、`rviz2` 依赖
 - 包名以 `_description` 结尾是社区惯例
-

讲次 8：创建 Launch 文件显示机器人模型

知识点

有了描述包，下一步是编写 Launch 文件，一键启动 `robot_state_publisher` 并在 RViz 2 中显示机器人。

`robot_state_publisher` 的角色

```
my_robot.urdf.xacro
|
|   ▼ xacro.process_file()
robot_description (URDF 字符串)
|
|   ▼ 作为参数传入
robot_state_publisher 节点
|
|   ── 发布 /robot_description 话题
|   ── 发布 /tf 和 /tf_static
|   ── 根据 joint_states 计算各 link 位姿
```

创建 `display.launch.py`

`launch/display.launch.py`

```

import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_description')

    # 1. 展开 xacro
    xacro_file = os.path.join(pkg_share, 'urdf', 'my_robot.urdf.xacro')
    robot_description = xacro.process_file(xacro_file).toxml()

    # 2. robot_state_publisher
    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        parameters=[{'robot_description': robot_description}],
    )

    # 3. joint_state_publisher_gui (手动拖动关节)
    joint_state_publisher_node = Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        name='joint_state_publisher_gui',
        output='screen',
    )

    # 4. RViz 2
    rviz_config = os.path.join(pkg_share, 'rviz', 'my_robot.rviz')
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', rviz_config],
    )

    return LaunchDescription([
        robot_state_publisher_node,
        joint_state_publisher_node,
        rviz_node,
    ])

```

启动与验证

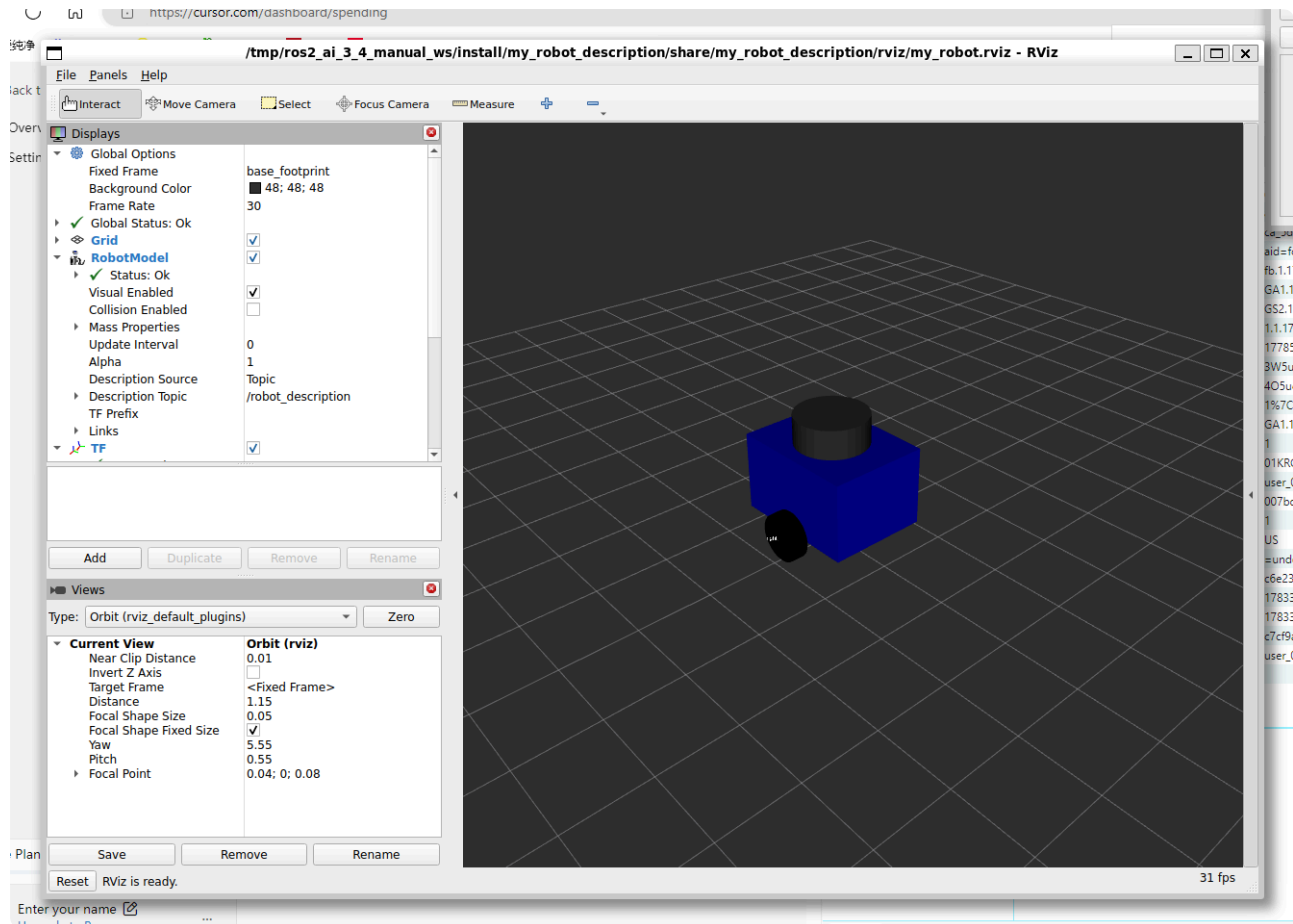
```

cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash
ros2 launch my_robot_description display.launch.py

```

预期效果： - RViz 2 窗口弹出 - 显示蓝色底盘 + 黑色双轮 + 激光雷达 - joint_state_publisher_gui 窗口可拖动轮子关节

真实运行效果如下，RViz 已自动加载 my_robot.rviz，Fixed Frame 为 base_footprint，RobotModel 状态为 OK：



检查 TF 树

```
# 新终端
ros2 run tf2_tools view_frames
# 生成带时间戳的 frames_*.pdf, 应看到:
# base_footprint → base_link → base_scan
#                               → left_wheel_link
#                               → right_wheel_link
```

检查话题

```
ros2 topic list | grep -E 'robot_description|joint_states|tf'
# 预期:
# /joint_states
# /robot_description
# /tf
# /tf_static
```

关键点

- Launch 文件用 `xacro.process_file()` 动态展开 xacro 为 URDF 字符串
 - `robot_state_publisher` 接收 `robot_description` 参数, 发布 TF 和 `robot_description` 话题
 - `joint_state_publisher_gui` 提供交互式关节控制 (开发调试必备)
 - 一条 `ros2 launch` 命令启动完整可视化流程
 - 用 `view_frames` 验证 TF 树结构
-

讲次 9：【Activity】在 Launch 文件中添加 RViz 配置

知识点

练习目标

创建 RViz 配置文件 `my_robot.rviz`，并在 Launch 文件中加载，实现启动即可看到完整机器人模型。

练习要求

1. 启动 `display.launch.py`（暂不带 rviz config）
2. 在 RViz 2 中手动添加以下 Display:
3. **Grid**: Reference Frame = `base_footprint`
4. **TF**: Show Names ✓、Show Axes ✓
5. **RobotModel**: Description Topic = `/robot_description`
6. 将 Fixed Frame 设为 `base_footprint`
7. 调整视角使机器人完整可见
8. File → Save Config As → `my_robot_description/rviz/my_robot.rviz`
9. 重新 build 并启动，确认配置自动加载

操作步骤

Step 1: 首次手动配置

```
ros2 launch my_robot_description display.launch.py
```

在 RViz 中：1. 左下角 Global Options → Fixed Frame → 选择 `base_footprint` 2. 点击 Add → By display type → Grid → OK 3. 点击 Add → TF → 勾选 Show Names 和 Show Axes 4. 点击 Add → RobotModel → Description Topic 保持 `/robot_description`

Step 2: 保存配置

File → Save Config As → 保存到：

```
~/ros2_ws/src/my_robot_description/rviz/my_robot.rviz
```

Step 3: 重新构建并验证

```
cd ~/ros2_ws
colcon build --packages-select my_robot_description
source install/setup.bash
ros2 launch my_robot_description display.launch.py
```

预期：RViz 自动加载 Grid + TF + RobotModel，无需手动添加。

RViz 配置关键项

Display	关键参数	推荐值
Global Options	Fixed Frame	<code>base_footprint</code>
Grid	Reference Frame	<code>base_footprint</code>
TF	Show Names	✓
TF	Show Axes	✓
RobotModel	Description Topic	<code>/robot_description</code>
RobotModel	Description Source	Topic

关键点

- RViz 配置文件 `.rviz` 保存 Display 布局，避免每次手动配置
 - 只检查静态机器人模型时可将 Fixed Frame 设为 `base_footprint`；导航或运动可视化通常选 `odom` 或 `map`
 - Launch 文件通过 `arguments=['-d', rviz_config]` 自动加载配置
 - `.rviz` 文件纳入 Git 版本管理，团队共享相同可视化环境
-

讲次 10: 【Activity】编写 Launch 文件 + 解答

知识点

练习目标

从零编写一个最小化的 `minimal_display.launch.py`，启动 `robot_state_publisher`、非 GUI 的 `joint_state_publisher` 和 `rviz2`（不含 `joint_state_publisher_gui`），并支持通过 Launch Argument 切换 xacro 文件。

练习要求

1. 创建 `launch/minimal_display.launch.py`
2. 支持 Launch Argument `model`，默认值为 `my_robot.urdf.xacro`
3. 启动 `robot_state_publisher`（传入 `robot_description`）
4. 启动非 GUI 的 `joint_state_publisher`，为左右轮 continuous joint 发布零位关节状态
5. 启动 `rviz2`（加载 `rviz/my_robot.rviz`）
6. 验证：`ros2 launch my_robot_description minimal_display.launch.py` 可正常显示

参考答案

`launch/minimal_display.launch.py`

```

import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, OpaqueFunction
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
import xacro

def launch_setup(context, *args, **kwargs):
    pkg_share = get_package_share_directory('my_robot_description')
    model = LaunchConfiguration('model').perform(context)
    xacro_path = os.path.join(pkg_share, 'urdf', model)
    robot_description = xacro.process_file(xacro_path).toxml()

    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        parameters=[{'robot_description': robot_description}],
        output='screen',
    )

    joint_state_publisher_node = Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
        name='joint_state_publisher',
        parameters=[{'robot_description': robot_description}],
        output='screen',
    )

    rviz_config = os.path.join(pkg_share, 'rviz', 'my_robot.rviz')
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        arguments=['-d', rviz_config],
        output='screen',
    )

    return [robot_state_publisher_node, joint_state_publisher_node, rviz_node]

def generate_launch_description():
    model_arg = DeclareLaunchArgument(
        'model',
        default_value='my_robot.urdf.xacro',
        description='Xacro file name in the urdf/ directory',
    )

    return LaunchDescription([
        model_arg,
        OpaqueFunction(function=launch_setup),
    ])

```

验证

```
colcon build --packages-select my_robot_description
source install/setup.bash
ros2 launch my_robot_description minimal_display.launch.py
```

```
# 新终端：确认节点运行
ros2 node list
# 预期：/robot_state_publisher, /joint_state_publisher, /rviz2

# 确认 robot_description 已发布
ros2 topic echo /robot_description --once | head -5
# 预期：data: '<?xml version="1.0" ?>...
```

关键点

- 对含非 fixed joint 的模型，最小可视化 Launch 需要 `joint_state_publisher` 发布默认关节状态
 - `DeclareLaunchArgument` 提供运行时配置灵活性
 - `xacro.process_file().toxml()` 是标准的 URDF 字符串生成方式
 - 先用最小 Launch 验证通，再逐步添加 `joint_state_publisher_gui` 等节点
-

讲次 11：创建 Launch 文件启动 Robot State Publisher

知识点

本节整合前几讲内容，创建一个可配置的模型检查 Launch 文件：包含 robot_state_publisher、joint_state_publisher_gui 和 RViz 2，并支持无头模式。

完整的 display.launch.py (最终版)

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition, UnlessCondition
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_description')

    # ===== Launch Arguments =====
    use_gui_arg = DeclareLaunchArgument(
        'use_gui',
        default_value='true',
        description='Start joint_state_publisher_gui for interactive joint control',
    )

    use_rviz_arg = DeclareLaunchArgument(
        'use_rviz',
        default_value='true',
        description='Start RViz2 with saved configuration',
    )

    # ===== 展开 Xacro =====
    xacro_path = os.path.join(pkg_share, 'urdf', 'my_robot.urdf.xacro')
    robot_description = xacro.process_file(xacro_path).toxml()

    # ===== 节点定义 =====
    robot_state_publisher_node = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        name='robot_state_publisher',
        output='screen',
        parameters=[{
            'robot_description': robot_description,
            'use_sim_time': False,
        }],
    )

    joint_state_publisher_gui_node = Node(
        package='joint_state_publisher_gui',
        executable='joint_state_publisher_gui',
        name='joint_state_publisher_gui',
        condition=IfCondition(LaunchConfiguration('use_gui')),
        output='screen',
        parameters=[{'robot_description': robot_description}],
    )

    joint_state_publisher_node = Node(
        package='joint_state_publisher',
        executable='joint_state_publisher',
```

```

        name='joint_state_publisher',
        condition=UnlessCondition(LaunchConfiguration('use_gui')),
        output='screen',
        parameters=[{'robot_description': robot_description}],
    )

    rviz_config = os.path.join(pkg_share, 'rviz', 'my_robot.rviz')
    rviz_node = Node(
        package='rviz2',
        executable='rviz2',
        name='rviz2',
        output='screen',
        arguments=['-d', rviz_config],
        condition=IfCondition(LaunchConfiguration('use_rviz')),
    )

    return LaunchDescription([
        use_gui_arg,
        use_rviz_arg,
        robot_state_publisher_node,
        joint_state_publisher_gui_node,
        joint_state_publisher_node,
        rviz_node,
    ])

```

Launch Argument 用法

```

# 默认: 启动 GUI + RViz
ros2 launch my_robot_description display.launch.py

# 无头模式: 启动 robot_state_publisher + 非 GUI joint_state_publisher
ros2 launch my_robot_description display.launch.py use_gui:=false use_rviz:=false

# 启动 robot_state_publisher + 非 GUI joint_state_publisher + RViz
ros2 launch my_robot_description display.launch.py use_gui:=false

```

robot_state_publisher 参数说明

参数	类型	说明
robot_description	string	URDF/XML 字符串 (必须)
use_sim_time	bool	是否使用仿真时钟 (Gazebo 中设为 true)
publish_frequency	double	TF 发布频率, 默认 50 Hz

数据流全图

```
display.launch.py
├── xacro.process_file(my_robot.urdf.xacro)
│   └── robot_description (string)
├── robot_state_publisher
│   ├── 订阅 /joint_states
│   ├── 发布 /robot_description (std_msgs/msg/String, transient_local QoS)
│   ├── 发布 /tf (动态关节)
│   └── 发布 /tf_static (fixed 关节)
├── joint_state_publisher_gui 或 joint_state_publisher
│   └── 发布 /joint_states (手动拖动关节或发布默认零位)
└── rviz2
    ├── 订阅 /robot_description → RobotModel Display
    └── 订阅 /tf + /tf_static → TF Display
```

无头模式（服务器/远程开发）

```
# 不启动任何 GUI
ros2 launch my_robot_description display.launch.py use_gui:=false use_rviz:=false

# 验证 TF 仍在发布
ros2 topic echo /tf_static --once
```

关键点

- 本节 Launch 支持 `use_gui` / `use_rviz` 条件启动，便于桌面调试和 CI 检查
 - `IfCondition` 根据 Launch Argument 决定是否启动某个节点
 - `use_sim_time` 在 Gazebo 仿真时设为 true（第四阶段）
 - `robot_state_publisher` 是 URDF 到 TF 的核心桥梁
 - 无头模式（无 GUI）适合远程服务器和 CI 测试
-

讲次 12：（总结）仿真准备 checklist

知识点

在进入 3.5 移动机器人完整 URDF 之前，确认以下仿真准备清单全部完成：

仿真准备 Checklist

#	检查项	验证命令	预期
1	Xacro 可正确展开	<code>xacro my_robot.urdf.xacro \\\ head -1</code>	<code><?xml version="1.0" ?></code>
2	URDF 结构合法	<code>check_urdf <(xacro ...)</code>	Successfully parsed
3	实体 link 有 collision	<code>xacro ... \\\ grep -c '<collision>'</code>	主模型 = 5
4	实体 link 有 inertial	<code>xacro ... \\\ grep -c '<inertial>'</code>	主模型 = 5
5	质量与惯性合理	对照几何公式和实物参数	张量正定，总质量约 1 kg
6	Gazebo 材质已配置	检查 gazebo_materials.xacro	每个 visual link 有对应 gazebo 标签
7	描述包可构建	<code>colcon build --packages-select my_robot_description</code>	无 error
8	Launch 可启动	<code>ros2 launch my_robot_description display.launch.py</code>	RViz 显示机器人
9	TF 树正确	<code>ros2 run tf2_tools view_frames</code>	base_footprint 为根
10	joint_states 发布	<code>ros2 topic echo /joint_states --once</code>	包含 wheel joint

关键点

- 仿真准备 = collision + inertial + gazebo 材质 + 描述包 + Launch
- 每项都有明确的验证命令，可自动化检查
- 完成 checklist 后，机器人描述已具备进入 Gazebo Harmonic 的基础；模型生成、驱动控制和传感器插件将在后续阶段配置
- 3.5 将在此基础上完成完整的移动机器人 URDF

本节总结

命令速查表

命令	说明
<code>ros2 launch my_robot_description display.launch.py</code>	一键可视化机器人
<code>ros2 topic echo /robot_description --once</code>	查看 URDF 字符串
<code>ros2 run tf2_tools view_frames</code>	生成 TF 树 PDF
<code>check_urdf file.urdf</code>	校验 URDF 结构
<code>xacro file.xacro -o file.urdf</code>	展开 Xacro

URDF 物理标签速查

标签	属性	Gazebo 必需
<code><visual></code>	外观几何 + 材质	RViz 必需
<code><collision></code>	碰撞几何	✅ 仿真必需
<code><inertial></code>	质量 + 惯性张量	✅ 仿真必需
<code><gazebo></code>	材质颜色 + 摩擦	✅ 仿真推荐

下一步

下一节 **3.5 P2 里程碑1：移动机器人 URDF** 中，我们将把所有知识点融会贯通，完成一个参考 TurtleBot3 Burger 的完整差速驱动移动机器人 URDF——包含底盘、双驱动轮、后万向轮、激光雷达，并通过 `check_urdf`、RViz 和 TF 树全面验证，交付项目二的第一个里程碑。